

# RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE SOFTWARE

**Disciplina:** DIM0501 - Boas Práticas de Programação

**Tema:** Aplicação dos conceitos de boas práticas em cenário simulado

**Semestre:** 2026.1

---

## 1. Identificação do Grupo

- **Integrante 1:** José Victor Onofre Sales
  - **Linguagem utilizada:** Java
  - **Link do repositório Git:** [https://github.com/josvictoronofre/Trabalho1\\_BoasPraticas](https://github.com/josvictoronofre/Trabalho1_BoasPraticas)
- 

## 2. Descrição do Sistema

- **O que o sistema faz:** O sistema gerencia um repositório de notícias em memória. Ele permite a inserção manual (onde o usuário define o texto e a categoria) e a inserção automática (onde o sistema analisa o comportamento do texto através de uma heurística de pontos e define se a notícia é confiável, duvidosa ou falsa). Além disso, realiza a listagem completa dos registros salvos no console.
  - **Qual era o objetivo do código original:** O objetivo original era servir como um script experimental simplificado (contido em um único arquivo de sistema) para testar a captura de strings e a estrutura condicional de um menu baseado em opções numéricas.
  - **Em que contexto ele poderia ser utilizado:** O sistema serve como um protótipo de backend ou microserviço de triagem para portais de comunicação, redes sociais ou ferramentas de moderação interna que precisam mitigar a propagação de desinformação (*Fake News*) antes de arquivar os dados.
-

## 3. Problemas Identificados

### 3.1 Problemas de Legibilidade

O fluxo principal do código original misturava a exibição visual de texto, a captura de dados do teclado e a lógica de negócios na mesma rotina. Além disso, havia variáveis genéricas que compartilhavam o mesmo bloco de execução do `switch`, poluindo a leitura do desenvolvedor.

### 3.2 Problemas de Organização

O código original era completamente monolítico. Não existia separação de responsabilidades. Uma única classe controlava o ciclo de vida do `Scanner` (infraestrutura), armazenava os dados diretamente no escopo do método principal (persistência), realizava os cálculos de pontuação (regras de negócio) e imprimia saídas de texto (interface).

### 3.4 Falta de Validação

O código original aceitava objetos inteiramente nulos ou strings compostas unicamente por espaços em branco para o campo de texto. Além disso, o sistema sofria de vulnerabilidade a falhas em tempo de execução: se o usuário digitasse uma letra onde se esperava um número inteiro no menu, a aplicação sofria um *crash* instantâneo ou entrava em loop infinito de erro.

### 3.5 Problemas de Documentação

O código inicial não apresentava nenhum comentário explicativo sobre o comportamento do buffer do teclado, tornando obscuro o motivo de certos comandos de leitura estarem posicionados antes de outros, além de carecer de metadados básicos de arquitetura.

---

## 4. Estratégias de Solução

### 4.1 Refatoração

- **O que foi renomeado/reajustado:** As variáveis de leitura de texto foram isoladas através de chaves `{ }` para delimitarem escopos locais próprios em cada `case`, eliminando colisões de nomenclatura.

- **Como as funções foram reorganizadas:** Métodos utilitários foram criados para isolar comportamentos específicos, retirando toda a carga procedural e algorítmica de dentro do método de inicialização `main`.

## 4.2 Modularização

O sistema foi dividido rigidamente em três módulos arquiteturais bem delimitados:

- **modelo:** Contendo a entidade fundamental `Noticia` (dados puros e construtores de classificação).
- **servico:** Contendo a classe `GerenciadorNoticias` (regras de negócio, lógica heurística e o `ArrayList` privado).
- **view:** Contendo a classe `SistemaNoticias` (responsável exclusiva pela interface CLI com o usuário).

## 4.3 Programação Defensiva

- **Quais validações foram adicionadas:** Foi inserida uma estrutura de *Guarda de Entrada* no método `adicionar` utilizando o predicado `!noticia.getTexto().isBlank()`, impedindo o salvamento de conteúdo sem sentido. O construtor passou a sanitizar entradas nulas transformando-as em strings vazias permanentes.
- **Como erros passaram a ser tratados:** Um bloco `try-catch` interceptando `InputMismatchException` passou a monitorar a entrada de dados do menu. Caso o usuário insira uma string inválida, o erro é contornado, uma mensagem amigável é impressa em tela de erro (`System.err`) e o buffer do teclado é esvaziado via `.nextLine()` para manter o sistema estável e contínuo.

## 4.4 Documentação

- **O que foi documentado:** Foram inseridos comentários explicativos de engenharia de software sobre polimorfismo (`@Override`), o uso estratégico de operadores ternários e a mecânica de limpeza do fluxo de entrada do sistema operacional.
- **Que tipo de comentários foram utilizados:** Comentários de linha única (`//`), posicionados imediatamente antes de linhas críticas do sistema (como limpezas de buffer e travas de interrupção).

---

# 5. Exemplos de Melhoria (Antes vs Depois)

## Exemplo 1: Nomenclatura, Encapsulamento e Semântica de Domínio

**Antes (Código Original Procedural e Genérico)**

Java

- class D {
- String t;
- String c;
- }
- 
- // Dentro da classe Sistema:
- public static void f(String a, String b) {
- if (a != null && !a.equals("")) {
- D d = new D();
- d.t = a;
- // ... atribuição exposta diretamente
- }
- }

### Depois (Código Refatorado com Padrões de Projetos O.O.)

Java

- // modelo/Noticia.java
- package modelo;
- 
- public class Noticia {
- private String texto;
- private String classificacao;
- 
- public Noticia(String texto, String classificacao) {
- this.texto = texto;
- // Sanitização e validação encapsuladas no construtor
- String classeTratada = (classificacao == null) ? "" :
- classificacao.trim().toLowerCase();
- }
- // Getters e Setters públicos controlando o acesso
- }

- **Explicação:** O código original utilizava uma classe genérica **D** com atributos **t** e **c** desprovidos de modificadores de acesso, permitindo a manipulação externa direta (quebra de encapsulamento). A refatoração transformou o registro na classe semântica **Noticia**, tornando os atributos estritamente privados (**private**) e transferindo a responsabilidade de validação e tratamento de strings diretamente para o construtor da classe.

---

### Exemplo 2: Otimização do Laço de Repetição e Polimorfismo

### Antes (Código Original com Iteração Indexada)

Java

- `public static void func2() {`
- `// lista tudo usando índice e acessando atributos diretamente`
- `for (int i = 0; i < data.size(); i++) {`
- `System.out.println("Texto: " + data.get(i).t);`
- `System.out.println("Classificacao: " + data.get(i).c);`
- `System.out.println("-----");`
- `}`
- `}`

### Depois (Código Refatorado com For-Each e Sobrescrita)

Java

- `// Na Camada View:`
- `for (Noticia noticia : gerenciador.getTodasNoticias()) {`
- `System.out.println(noticia); // Invoca implicitamente o toString() polimórfico`
- `}`
- 
- `// Na Entidade Noticia (modelo):`
- `@Override`
- `public String toString() {`
- `return "Texto: " + texto + "\nClassificacao: " + classificacao + "\n";`
- `}`

- **Explicação:** A função original `func2` exibiu forte acoplamento ao varrer o array por índices (`data.get(i)`) e extrair as propriedades textuais brutas diretamente na tela. Na versão melhorada, aplicou-se o laço `for-each` para maior legibilidade e utilizou-se o polimorfismo através da sobrescrita do método `@Override toString()`, delegando à própria classe `Noticia` a responsabilidade de saber como se renderizar textualmente.

---

## Exemplo 3: Controle do Fluxo do Menu e Resiliência a Falhas

### Antes (Código Original com Condicionais em Cascata)

Java

- `String op = sc.nextLine();`
-

- `if (op.equals("1")) {`
- `addManual(sc);`
- `} else if (op.equals("2")) {`
- `addAuto(sc);`
- `} else if (op.equals("3")) {`
- `func2();`
- `} // ... controle manual baseado puramente em Strings ...`

### Depois (Código Atual com Inteiros, Switch-Case e Captura de Erros)

Java

- `try {`
  - `opcao = scanner.nextInt(); // Captura tipada numericamente`
  - `switch (opcao) {`
  - `case 1: {`
  - `// Travas de interrupção contra fall-through`
  - `break;`
  - `}`
  - `case 3:`
  - `for (Noticia noticia : gerenciador.getTodasNoticias()) { ... }`
  - `break;`
  - `}`
  - `} catch (InputMismatchException e) {`
  - `System.err.println("Erro! valor inserido invalido para o campo!");`
  - `scanner.nextLine(); // Limpeza preventiva do buffer do teclado`
  - `}`
- 
- **Explicação:** No programa original, o menu tratava a entrada de dados estritamente como `String` com sucessivos blocos `if-else` procedurais. A versão final adotou o `switch-case` com blocos de escopo delimitados por chaves `{}` e adicionou robustez à interface através de programação defensiva com um bloco `try-catch`, protegendo o sistema contra entradas de caracteres inválidos no terminal e limpando o buffer do teclado para evitar loops de travamento.
- 

## 6. Organização Final do Código

A estrutura de diretórios do projeto foi organizada seguindo a separação estrita de responsabilidades:

Plaintext

- src/
  - └─ arquivo\_original
  - └─ Sistema.java           # Arquivo original intocado
  - └─ modelo/
  - └─ Noticia.java           # Entidade de dados com regras de construtor e toString()
  - └─ servico/
  - └─ GerenciadorNoticias.java # Regras de negócio, Heurística de análise e Repositório
  - └─ view/
  - └─ SistemaNoticias.java   # Classe principal, Menu interativo e captura de Inputs
- 
- **modelo:** Modela o domínio do problema. Garante que nenhuma notícia seja instanciada em estado corrompido ou inconsistente.
  - **servico:** Centraliza a inteligência do software. Gerencia o estado e o ciclo de armazenamento das notícias isolado do ambiente externo.
  - **view:** Atua na borda do sistema. Cuida puramente da formatação visual e recepção bruta de dados, encaminhando as demandas para os métodos adequados de serviço.
- 

## 7. Conclusão

- **O que o grupo aprendeu com o trabalho:** O grupo aprendeu que escrever código funcional não é sinônimo de escrever software sustentável. Compreendemos a real necessidade de isolar responsabilidades e a importância vital de gerenciar fluxos de entrada de recursos compartilhados da máquina, como o buffer do teclado.
- **Quais foram as maiores dificuldades:** A maior dificuldade técnica centrou-se em mapear o comportamento sutil do **Scanner** do Java, especificamente o entendimento de como resíduos de quebra de linha (**\n**) permanecem na pilha do sistema e como tratá-los de dentro de blocos de captura de erro.
- **Como as boas práticas impactaram o código:** O impacto foi transformador. O código evoluiu de um script experimental e quebrável para um sistema coeso, limpo, modular, resiliente e extensível, pronto para receber novas modificações profissionais sem risco de efeitos colaterais ocultos.